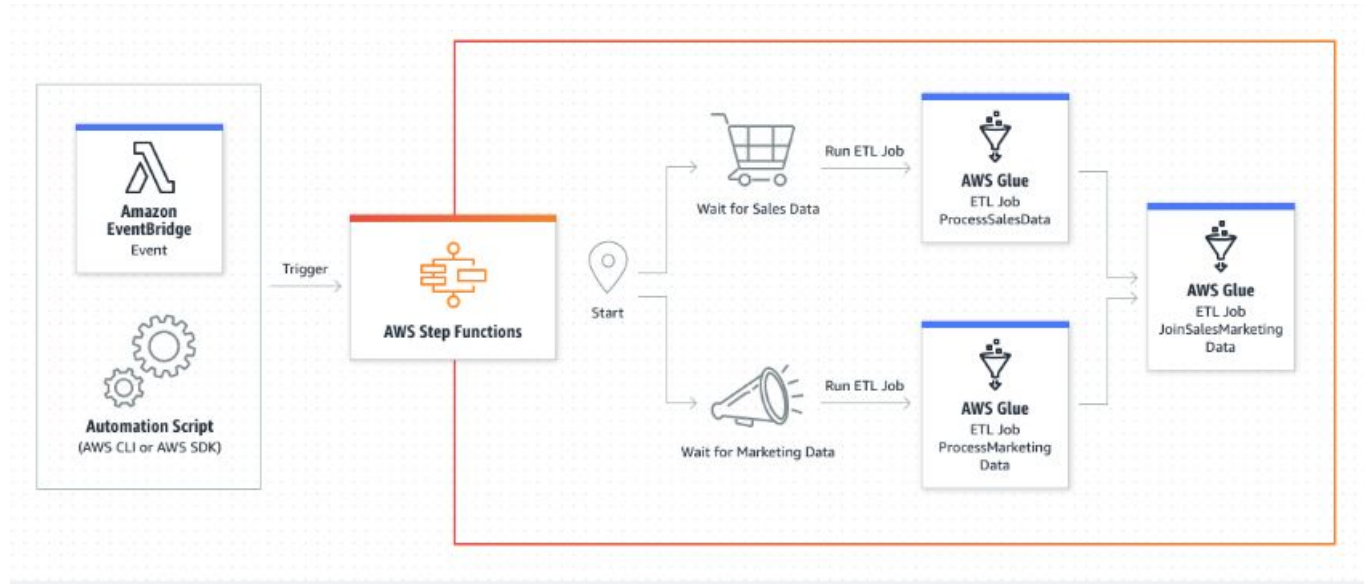


AWS Step Function

- **Overview:** AWS Step Functions is a serverless orchestration service that makes it easy to sequence AWS services into business-critical applications. Through visual workflows, you can design and run complex business processes, automations, and data pipelines.
- **How It Works:** Step Functions lets you coordinate multiple AWS services into serverless workflows so you can build and update apps quickly. You define your **workflows** as **state machines**, with each state representing a step in your application that can do work (Task states), decide which path to take (Choice states), stop with an error (Fail states), and more.



- **Key Features:**

- ***Visual Workflow Management:*** A graphical interface to visualize the components of your application as a series of steps.
- ***Serverless Nature:*** Automatically triggers and manages each step in your workflow, scaling with no need for provisioning infrastructure.
- ***Reliability and Fault Tolerance:*** Built-in error handling, retry logic, and parallel execution paths ensure reliable application execution.
- ***Integration with AWS Ecosystem:*** Seamlessly integrates with AWS services like Lambda, ECS, SNS, SQS, and DynamoDB, enabling comprehensive applications.

- **Components Of Step Function:**

- ***State Machine:*** The core component that defines the workflow or orchestration logic in JSON format. It consists of states that represent the workflow steps, with each state performing a specific function within the orchestration.
- ***States:*** Building blocks of a state machine, each representing a single unit of work or a decision point. Types of states include:
 - **Task State:** Represents a single unit of work, such as invoking an AWS Lambda function, publishing to an SNS topic, or inserting an item into DynamoDB.
 - **Choice State:** Adds branching logic, allowing the workflow to choose different paths based on input.
 - **Wait State:** Delays the state machine from transitioning to the next state for a specified time.
 - **Parallel State:** Executes multiple branches of tasks in parallel, merging their results upon completion.
 - **Succeed/Fail States:** Indicate the successful or unsuccessful termination of the state machine.
 - **Pass State:** Passes its input to its output or injects fixed data, useful for parameter passing or static configuration.
 - **Map State:** Processes a list of input data by iterating through each item and applying a set of steps.

- **Key Features:**

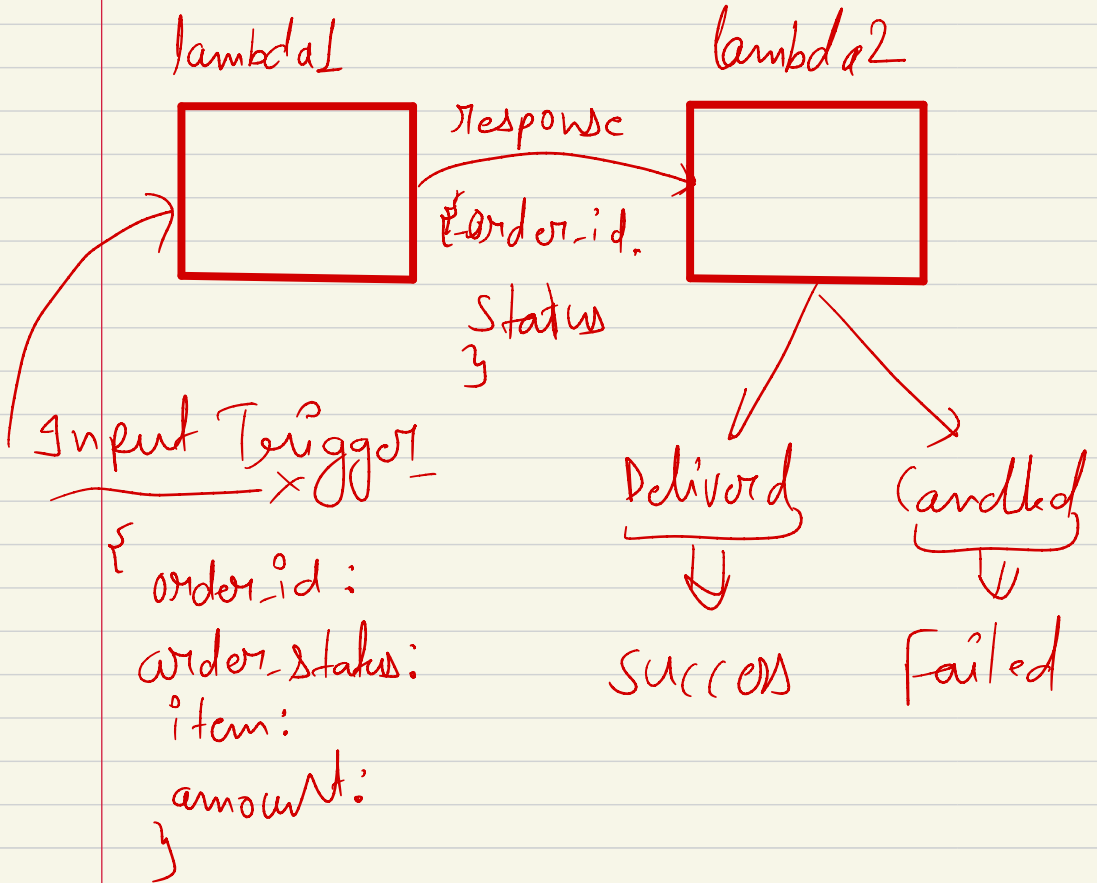
- ***Visual Workflow Management:*** A graphical interface to visualize the components of your application as a series of steps.
- ***Serverless Nature:*** Automatically triggers and manages each step in your workflow, scaling with no need for provisioning infrastructure.
- ***Reliability and Fault Tolerance:*** Built-in error handling, retry logic, and parallel execution paths ensure reliable application execution.
- ***Integration with AWS Ecosystem:*** Seamlessly integrates with AWS services like Lambda, ECS, SNS, SQS, and DynamoDB, enabling comprehensive applications.

- **Components Of Step Function:**

- ***State Machine:*** The core component that defines the workflow or orchestration logic in JSON format. It consists of states that represent the workflow steps, with each state performing a specific function within the orchestration.
- ***States:*** Building blocks of a state machine, each representing a single unit of work or a decision point. Types of states include:
 - **Task State:** Represents a single unit of work, such as invoking an AWS Lambda function, publishing to an SNS topic, or inserting an item into DynamoDB.
 - **Choice State:** Adds branching logic, allowing the workflow to choose different paths based on input.
 - **Wait State:** Delays the state machine from transitioning to the next state for a specified time.
 - **Parallel State:** Executes multiple branches of tasks in parallel, merging their results upon completion.
 - **Succeed/Fail States:** Indicate the successful or unsuccessful termination of the state machine.
 - **Pass State:** Passes its input to its output or injects fixed data, useful for parameter passing or static configuration.
 - **Map State:** Processes a list of input data by iterating through each item and applying a set of steps.



Scenario



Project Assignment: Building an Incremental Data Processing Pipeline with AWS

Objective:

Develop a data engineering solution that simulates the real-world scenario of processing sales transaction data. This project focuses on implementing an incremental data ingestion pipeline into Amazon Redshift, using AWS Step Functions for orchestration. The pipeline will handle CSV files arriving in an S3 bucket, process the data with AWS Lambda, and perform upsert operations in Redshift to maintain the most current dataset.

Project Scope:

- **Data Description:** You will work with a sales transactions dataset that includes details such as transaction ID, date, product ID, quantity, price, and store location.
- **Data Ingestion:** CSV files will be simulated to arrive in an S3 bucket. Each file represents new transactions that may include new records or updates to existing records based on the transaction ID.

Tasks:

- **Data Modeling:**
 - Design a table schema for the sales transactions in Amazon Redshift. Consider how you will handle the upsert operation for new and existing records.
- **Mock Data Generation:**
 - Write a script to generate mock CSV files for the sales transactions dataset. Each file should contain multiple records of sales transactions. Consider using Python for script development.
 - Simulate the arrival of these CSV files into an S3 bucket. You may manually upload them or automate this process as an extension of your script.
 - Sample CSV Data

```
TransactionID,Date,ProductID,Quantity,Price,StoreLocation
1,2024-04-01,1001,2,20.00,New York
2,2024-04-01,1002,1,35.00,Los Angeles
3,2024-04-02,1003,1,15.00,Chicago
4,2024-04-02,1001,1,20.00,New York
5,2024-04-03,1004,2,40.00,Houston
```

- **AWS Step Functions Workflow:**
 - Design a state machine in AWS Step Functions that orchestrates the following tasks:

- Triggering a Lambda function when a new CSV file is uploaded to S3.
- Processing the CSV file with the Lambda function to read its contents.
- Inserting new records or updating existing records in the Amazon Redshift table (upsert operation).

- **Lambda Function for Data Processing:**

- Develop a Lambda function in Python that is triggered by S3 events. This function should:
 - Read the incoming CSV file from S3.
 - Parse the CSV file and transform the data if necessary.
 - Connect to Amazon Redshift to perform the upsert operations. Consider using the psycopg2 library or a similar tool for database interactions.

- **Data Upsert in Amazon Redshift:**

- Implement the logic for data upsert (insert or update) in Redshift. Ensure that:
 - New records are inserted.
 - Existing records (based on the transaction ID) are updated with new information.

- **Documentation:**

- Document your workflow design, including the Step Functions state machine, Lambda function logic, and Redshift upsert strategy.
- Provide a brief explanation of your mock data generation process and the structure of your Redshift table schema.

Deliverables:

- Python script for mock data generation.
- AWS Step Functions state machine definition (JSON format).
- Source code for the AWS Lambda function.
- SQL script for the Redshift table schema and upsert logic.
- A comprehensive documentation file explaining your solution and design choices.

Interview Questions & Answer

Q.) How does AWS Step Functions handle error handling and retries?

A.) AWS Step Functions allow for robust error handling within your state machine. You can define retries on tasks that might fail due to transient issues and catch blocks to handle known errors. The Retry field specifies settings like the number of retry attempts, the interval between retries, and backoff rates. The Catch field allows you to redirect the workflow to a specific state when an error occurs, enabling graceful failure handling or alternative processing paths.

Q.) Explain the difference between Standard and Express Workflows in AWS Step Functions.

A.) Standard Workflows offer exactly-once execution with a duration of up to one year, making them suitable for long-running, durable, and auditable workflows. They support features like wait states for long delays and human activities requiring manual intervention. Express Workflows are designed for high-volume, short-duration (up to 5 minutes), event-processing workloads. They provide at-least-once execution and are billed by the number of state transitions, making them cost-effective for high-throughput applications.

Q.) What is the maximum execution duration for a single execution in AWS Step Functions, and how can you manage long-running tasks?

A.) A single execution in Standard Workflows can run for up to one year. For long-running tasks, you should design your state machine to include checkpointing and error handling to manage and maintain the state throughout the execution. For tasks expected to run beyond the service limits, you might implement a continuation pattern by triggering a new execution from the current one before it ends.

Q.) How can you integrate AWS Lambda functions and other AWS services with AWS Step Functions?

A.) AWS Step Functions allows integration with AWS Lambda functions directly through the "Task" state, where you define the Lambda function ARN and the input it requires. For other AWS services, Step Functions supports the Amazon States Language (ASL) service integration patterns, enabling you to directly call and pass parameters to various

AWS services like Amazon S3, Amazon ECS, Amazon SNS, Amazon SQS, and more, without needing to write custom integration code.

Q.) Describe how you would implement dynamic parallelism in a workflow using AWS Step Functions.

A.) Dynamic parallelism can be implemented using the Map state, which enables you to process a number of inputs in parallel. The Map state dynamically iterates over an array of items and executes a sub-workflow for each item concurrently. This is particularly useful for scenarios where you need to perform operations like batch processing of items where each item can be processed independently.

Q.) Can you explain the concept of a "callback pattern" in AWS Step Functions?

A.) The callback pattern allows an external process or service to resume a paused Step Functions execution. This is achieved using the Task state with the waitForResource integration pattern. Step Functions generate a task token that is passed to an external service or process, which then performs some work outside of Step Functions. Once the external task is complete, the service or process returns the task token to signal completion, and the Step Functions execution resumes from where it was paused.

Q.) How do you secure sensitive data in AWS Step Functions' state inputs and outputs?

A.) To secure sensitive data in state inputs and outputs, you can use AWS Key Management Service (KMS) to encrypt the state machine definition and the data passed through it. Additionally, you can implement IAM policies to restrict access to the state machine, and utilize CloudWatch Logs with encryption to log the execution history securely. Also, for highly sensitive data, consider minimizing the amount of sensitive data passed between states or using external secure storage to hold sensitive data, fetching it when necessary.

Q.) What is the maximum payload size for inputs and outputs in AWS Step Functions, and how can you handle larger payloads?

A.) The maximum payload size for inputs and outputs in AWS Step Functions is 256KB. For larger payloads, you should use Amazon S3. You can store the payload in an S3 bucket and pass the S3 object reference (e.g., the bucket name and object key) as the input or output. This way, the state machine only handles references to the data, not the data itself, effectively bypassing the size limit.

Q.) Explain how you would monitor and troubleshoot a failing execution in AWS Step Functions.

A.) To monitor and troubleshoot failing executions, you can use CloudWatch Metrics and Logs. Metrics provide operational health insights, like execution counts, failures, and throttles, while Logs capture detailed execution history and state transitions, which can be invaluable for debugging. AWS Step Functions also offers a visual interface to trace the path of an execution, allowing you to see which states succeeded and where failures occurred, facilitating easier identification of issues.

Q.) How does AWS Step Functions manage state transitions, and what are the implications for data passing between states?

A.) AWS Step Functions manages state transitions according to the Amazon States Language, where outputs from one state can become inputs to the next. The service ensures that data is passed between states in a reliable manner, but developers must carefully design state inputs and outputs to ensure that necessary data is preserved across transitions. The ResultPath parameter allows you to control how the output of a state is combined with the original input, enabling precise management of state data.

Q.) Design a high-throughput data processing pipeline for processing web server logs stored in S3. Use AWS Lambda for parsing and transforming logs, Amazon Kinesis Firehose for buffering and batch processing, and Amazon Redshift for analytics and querying.

A.)

- **Data Ingestion:** Web server logs are continuously uploaded to an S3 bucket. Configure S3 event notifications to trigger a Lambda function upon the arrival of new logs.
- **Log Parsing Lambda Function:** The triggered Lambda function parses the raw log data, extracts relevant fields (e.g., request time, status code, user agent), and transforms the data into a structured format suitable for analytics.
- **Kinesis Firehose for Buffering:** After parsing, the Lambda function publishes the structured logs to an Amazon Kinesis Firehose delivery stream. Kinesis Firehose buffers the logs, aggregating them into larger batches to optimize the loading process into Redshift.
- **Data Storage in Redshift:** Kinesis Firehose loads the aggregated logs into an Amazon Redshift cluster, utilizing the COPY command for efficient bulk inserts.

This step may involve intermediate storage in an S3 staging area, configured within Kinesis Firehose.

- **Step Functions Workflow:** Use AWS Step Functions to orchestrate the workflow:
 - **State 1:** Trigger the Log Parsing Lambda function upon log file upload to S3.
 - **State 2:** Error handling state to catch any parsing failures and notify administrators via SNS.
 - **State 3:** Success state that logs the successful processing and storage of log data.

Q.) Create an event-driven system for periodic database cleanup in DynamoDB and migrating historical data to S3 for long-term storage. Use AWS Step Functions for orchestration, Lambda for data processing, and Amazon Glue for data migration tasks.

A.)

- **Trigger Mechanism:** Configure a CloudWatch Events rule to trigger the Step Functions state machine on a scheduled basis (e.g., nightly).
- **Step Functions State Machine:**
 - **State 1:** A Lambda function scans the DynamoDB table for items matching cleanup criteria (e.g., older than a year).
 - **State 2:** For items marked for cleanup, the same or another Lambda function archives the items to S3. This involves serializing the items and storing them in a structured format (e.g., JSON, Parquet) in an S3 bucket.
 - **State 3:** After archiving, another Lambda function deletes the archived items from DynamoDB to complete the cleanup process.
 - **State 4:** Use AWS Glue to catalog the archived data in S3, allowing for easier querying and analysis later.
 - **Error Handling:** Implement error handling states to manage any failures in the archive or delete processes, including notification of administrators via SNS.

Q.) Design a real-time analytics pipeline for e-commerce transaction data, using Amazon Kinesis for real-time data ingestion, AWS Lambda for initial processing, Amazon S3 for raw data storage, and Amazon Redshift for aggregation and querying.

A.)

- **Data Ingestion:** Configure Amazon Kinesis Streams to ingest e-commerce transaction data in real-time.
- **Initial Processing with Lambda:** Set up a Lambda function triggered by Kinesis to perform initial processing on the transaction data, such as validation, enrichment (e.g., adding product information from a DynamoDB table), and formatting.
- **Raw Data Storage:** The Lambda function then stores the processed data in an S3 bucket for durability and later analysis. This step can involve organizing data into partitions based on transaction date to optimize for query performance.
- **Load into Redshift:** Use Amazon Kinesis Firehose, configured to read from the processed stream in S3, to batch and load data into an Amazon Redshift cluster. This setup leverages Redshift's COPY command for efficient data loading.
- **Step Functions Workflow:**
 - **State 1:** Trigger Lambda functions for initial data processing upon data arrival in Kinesis Streams.
 - **State 2:** On successful processing, proceed to store the data in S3.
 - **State 3:** Configure a batch load operation into Redshift via Kinesis Firehose.
 - **Error Handling:** Implement states to manage and log processing or loading errors, potentially triggering retry mechanisms or notifying via SNS for manual intervention.

Q.) Set up an automated disaster recovery backup system for critical data stored in DynamoDB, including regular backups to S3 and cross-region replication for high availability.

A.)

- **Regular Backups:** Utilize AWS Lambda functions to create backups of DynamoDB tables. These backups can be triggered by AWS Step Functions on a schedule defined in CloudWatch Events.
- **Backup to S3:** After creating a backup, the Lambda function uploads the backup files to an S3 bucket. Use S3's versioning and lifecycle policies to manage these backups, including transitioning older backups to S3 Glacier for cost-effective long-term storage.
- **Cross-Region Replication:** Configure S3 cross-region replication to replicate backup files to another AWS region, ensuring data availability in case of a regional outage.
- **Step Functions Workflow:**

- **State 1:** Schedule the backup process using CloudWatch Events to trigger the Step Functions state machine.
- **State 2:** Execute Lambda functions to backup DynamoDB tables and store the backup in S3.
- **State 3:** Verify backups and manage cross-region replication settings.
- **Error Handling:** Implement states for error catching, including logging errors and sending notifications through SNS for any issues encountered during the backup process.

Q.) Design a serverless video processing pipeline that triggers when new videos are uploaded to S3, uses Lambda to initiate processing jobs in Amazon Elastic Transcoder, and stores the output in a different S3 bucket. Utilize Step Functions to manage the workflow and handle processing statuses.

A.)

- **Trigger:** An S3 event notification triggers a Step Functions execution when a new video is uploaded.
- **Step Functions Workflow:**
 - **State 1:** A Lambda function is invoked to submit a video transcoding job to Amazon Elastic Transcoder, using video settings predefined for the target device or platform.
 - **State 2:** Polling state to check the status of the transcoding job. This can be implemented as a loop within Step Functions, periodically invoking a Lambda function to query the Elastic Transcoder's job status.
 - **State 3:** Once transcoding is complete, another Lambda function moves the transcoded video from the Elastic Transcoder's output bucket to the final S3 bucket. This state could also involve updating a database record in DynamoDB to reflect the availability of the new video.
 - **Error Handling:** Implement error handling to catch any failures in transcoding or file movement, including retry strategies or notifications through SNS.

Q.) Build a fraud detection system for financial transactions using Kinesis Data Streams for real-time transaction ingestion, Lambda for initial fraud detection logic, and Step Functions to orchestrate the detection workflow, including notification and transaction blocking actions.

A.)

- **Data Ingestion:** Financial transactions are streamed in real-time through Amazon Kinesis Data Streams.
- **Initial Fraud Detection:** A Lambda function subscribed to the Kinesis stream applies initial fraud detection algorithms on the transactions, marking suspicious ones.
- **Step Functions Workflow:**
 - **State 1:** Trigger the Lambda function for initial fraud detection.
 - **State 2:** For transactions marked as suspicious, invoke a detailed analysis Lambda function that uses a more comprehensive set of fraud detection criteria.
 - **State 3:** Based on the outcome of the detailed analysis, execute decision states that either block the transaction, flag it for manual review, or let it proceed. This could involve updating a DynamoDB table with the transaction status and sending notifications through SNS.
 - **Error Handling:** Implement robust error handling for each step, ensuring that any failures in processing or decision-making are logged and alerted for investigation.

Q.) Develop a data aggregation and reporting pipeline that collects data from multiple sources (S3, DynamoDB), aggregates and processes the data using AWS Glue, and generates reports stored in Amazon Redshift for BI tools access.

A.)

- **Data Collection:** Use AWS Lambda functions to collect data from various sources and store it in a centralized S3 bucket in a raw data format.
- **Data Aggregation with AWS Glue:** Trigger an AWS Glue ETL job from Step Functions to transform, clean, and aggregate the collected data. The job reads from the S3 bucket, performs necessary transformations, and writes the aggregated data to an Amazon Redshift cluster.
- **Step Functions Workflow:**
 - **State 1:** Collection states where Lambda functions are invoked to fetch and store data in S3.
 - **State 2:** An AWS Glue job state for data transformation and aggregation.
 - **State 3:** A final state to handle the loading of transformed data into Redshift, preparing it for analysis and reporting.
 - **Error Handling:** Define error handling states for each step, ensuring failures in data collection, processing, or loading trigger appropriate notifications and logging.

Q.) Create a system to manage IoT device data, including ingestion, storage, batch processing for analytics, and real-time monitoring. Use IoT Core for data ingestion, S3 for storage, AWS Glue for batch processing, and Kinesis for real-time data streaming.

A.)

- **Ingestion:** IoT device data is ingested through AWS IoT Core and streamed to both Amazon S3 for storage and Amazon Kinesis for real-time processing.
- **Batch Processing:** Schedule AWS Glue ETL jobs through Step Functions to run periodic analytics on the data stored in S3, transforming and loading the results into Amazon Redshift for querying and analysis.
- **Real-Time Monitoring:** Use a Lambda function to process the real-time stream from Kinesis, applying real-time analytics or anomaly detection, and updating a DynamoDB table with the latest device statuses.
- **Step Functions Workflow:**
 - **State 1:** Data ingestion coordination, ensuring that IoT data is correctly routed to S3 and Kinesis.
 - **State 2:** Trigger AWS Glue ETL jobs for batch processing of historical data.
 - **State 3:** Manage the real-time processing Lambda function, ensuring continuous operation and error handling.
 - **Error Handling:** Include states for error logging, retry mechanisms, and alerting to handle failures in data ingestion, processing, or analysis steps.

Q.) Implement a content moderation system that automatically processes and moderates user-generated content uploaded to S3, using Amazon Rekognition for image and video moderation, and Lambda for text moderation. Use Step Functions to orchestrate the moderation workflow and DynamoDB to track moderation status.

A.)

- **Trigger:** An S3 event triggers a Step Functions execution when new content is uploaded by users.
- **Moderation Workflow:**
 - **State 1:** A Lambda function determines the content type (image, video, text) and routes it to the appropriate moderation service: Amazon Rekognition for images and videos, and a custom Lambda function for text analysis.

- **State 2:** Based on moderation results, decision states in Step Functions route content to either be published, flagged for manual review, or rejected.
- **State 3:** Update a DynamoDB table with the content moderation status and results for each piece of user-generated content.
- **Error Handling:** Implement comprehensive error handling to manage failures in content analysis, including retry strategies and notifications for manual intervention.

Q.) Design a scalable web crawling system that dynamically allocates resources based on the queue of URLs to be crawled. Use SQS for URL queue management, Lambda for executing crawl tasks, and Step Functions to orchestrate scaling decisions and processing flows.

A.)

- **URL Queue Management:** Store a queue of URLs to be crawled in an Amazon SQS queue. This allows for easy scaling and management of crawl tasks.
- **Dynamic Resource Allocation:** Use Step Functions to monitor the SQS queue size and make scaling decisions, triggering additional Lambda functions for increased crawling capacity when the queue size exceeds a certain threshold.
- **Crawl Task Execution:** Lambda functions dequeue URLs from SQS and perform the web crawl tasks, parsing content and storing relevant data in DynamoDB or S3, depending on the content type and processing requirements.
- **Step Functions Workflow:**
 - **State 1:** Monitoring state to assess the SQS queue size and decide on scaling.
 - **State 2:** Task execution state where Lambda functions are dynamically triggered based on the required scale.
 - **State 3:** Post-processing state to aggregate crawl results, update databases, and potentially trigger further crawling tasks based on the discovered links.
 - **Error Handling:** Design error handling states for retrying failed crawl tasks, managing malformed URLs, and ensuring robust logging and notification mechanisms for operational issues.

← Test your knowledge : Class_Quiz

Quiz . 15 Total Points

😊 Great job! Keep pushing for that perfect score! 🚀🌟

Your marks : 12 / 15

1. What is AWS Step Functions primarily used for?

- ☐ Managing AWS Lambda functions only
- ☐ Automated deployment of AWS resources
- ☒ Coordinating multiple AWS services into serverless workflows
- ☐ Load balancing across multiple EC2 instances

✔ **Correct!**

2. Which of the following is NOT a type of state in an AWS Step Functions state machine?

- ☐ Choice State
- ☐ Wait State
- ☐ Loop State
- ☒ Fail State

2. Which of the following is NOT a type of state in an AWS Step Functions state machine?

- ☐ Choice State
- ☐ Wait State
- ☐ Loop State
- ☒ Fail State

✗ **Incorrect!**

Loop State is the correct answer

3. In AWS Step Functions, what JSON-based language is used to define the state machines?

- ☐ JavaScript
- ☒ Amazon State Language (ASL)
- ☐ Python
- ☐ AWS Markup Language (AWML)

✓ **Correct!**

4. Which AWS service is NOT directly integrable with AWS Step Functions?

4. Which AWS service is NOT directly integrable with AWS Step Functions?

- ☐ AWS Lambda
- ☐ Amazon S3
- ☒ Amazon EC2
- ☐ Amazon SES

✓ Correct!

5. How does AWS Step Functions maintain the application's state?

- ☐ By using Amazon DynamoDB
- ☒ Through an internal state engine
- ☐ By storing it in Amazon S3
- ☐ Using Amazon RDS

✓ Correct!

6. What is the maximum duration a single execution of a Step Function state machine can run?

6. What is the maximum duration a single execution of a Step Function state machine can run?

- ☐ 5 minutes
- ☐ 1 hour
- ☐ 24 hours
- ☒ 1 year

✓ Correct!

7. Which feature allows AWS Step Functions to directly call and pass parameters to API actions of AWS services?

- ☒ Service Integration
- ☐ Lambda Integration
- ☐ Direct API
- ☐ AWS SDK Integration

✓ Correct!

8. In AWS Step Functions, what is the purpose of a 'Map' state?

- ☐ To execute a task multiple times in parallel

8. In AWS Step Functions, what is the purpose of a 'Map' state?

- ☐ To direct the workflow to multiple paths
- ☒ To iterate over a collection of items and execute a set of steps for each item
- ☐ To pause the state machine
- ☐ To map domain names to state machines

✓ Correct!

9. How can you secure sensitive data in your state machine definitions?

- ☐ By using AWS IAM roles
- ☒ Encrypting data using AWS KMS
- ☐ Using VPC Endpoints
- ☐ By enabling AWS Shield

✓ Correct!

10. What is the maximum size for the input or output data of a step in AWS Step Functions?

- ☐ 256 KB

10. What is the maximum size for the input or output data of a step in AWS Step Functions?

- ☐ 32 KB
- ☐ 256 KB
- ☐ 1 MB
- ☒ 256 MB

✗ **Incorrect!**

256 KB is the correct answer

11. Which of the following best describes the 'Parallel' state in AWS Step Functions?

- ☐ Executes all branches sequentially
- ☒ Runs multiple branches of steps concurrently
- ☐ Divides the input data into smaller chunks for parallel processing
- ☐ Executes different state machines in parallel

✓ **Correct!**

12. When using AWS Step Functions, what is a best practice for handling errors?

11. Which of the following best describes the 'Parallel' state in AWS Step Functions?

- ☐ Executes all branches sequentially
- ☒ Runs multiple branches of steps concurrently
- ☐ Divides the input data into smaller chunks for parallel processing
- ☐ Executes different state machines in parallel

✓ Correct!

12. When using AWS Step Functions, what is a best practice for handling errors?

- ☐ Use a single Catch block for all states
- ☐ Handle errors in the client application
- ☒ Specify Retry and Catch fields on states that can fail
- ☐ Ignore errors to maintain state machine performance

✓ Correct!

13. What happens if a step in an AWS Step Functions state machine fails?

- ☐ The state machine continues to execute the next step

13. What happens if a step in an AWS Step Functions state machine fails?

- ☐ The entire state machine stops and rolls back
- ☐ Execution continues to the next step
- ☐ Execution stops, and an error is reported
- ☒ Automatically retries the failed step

✗ **Incorrect!**

Execution stops, and an error is reported is the correct answer

14. Which of the following is NOT a valid field in the Amazon State Language?

- ☐ "StartAt"
- ☐ "Retry"
- ☒ "Transform"
- ☐ "Comment"

✓ **Correct!**

- ☐ "Retry"
- ☒ "Transform"
- ☐ "Comment"

✔ Correct!

15. AWS Step Functions integrates with AWS X-Ray for what purpose?

- ☐ To monitor the live status of your workflows
- ☐ To encrypt data in transit
- ☒ To trace and analyze requests made by your state machine
- ☐ To automatically deploy your state machines across multiple regions

✔ Correct!

Submit